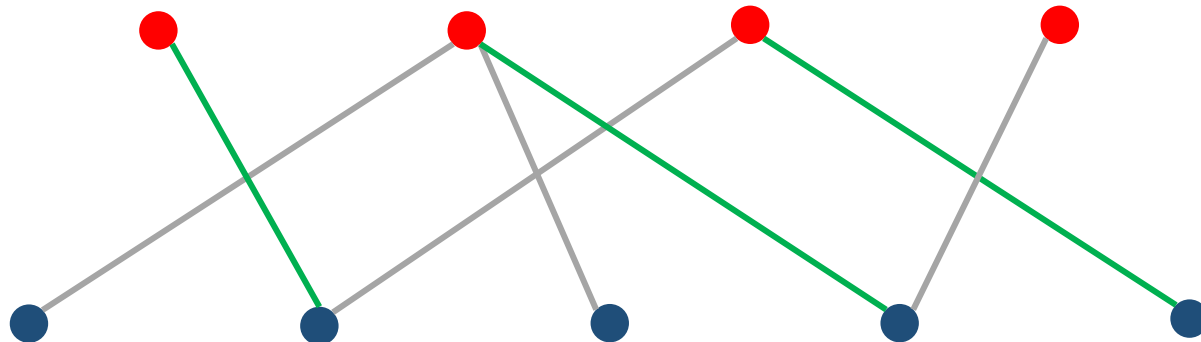


Паросочетания
Алгоритм Куна
Задача о назначениях

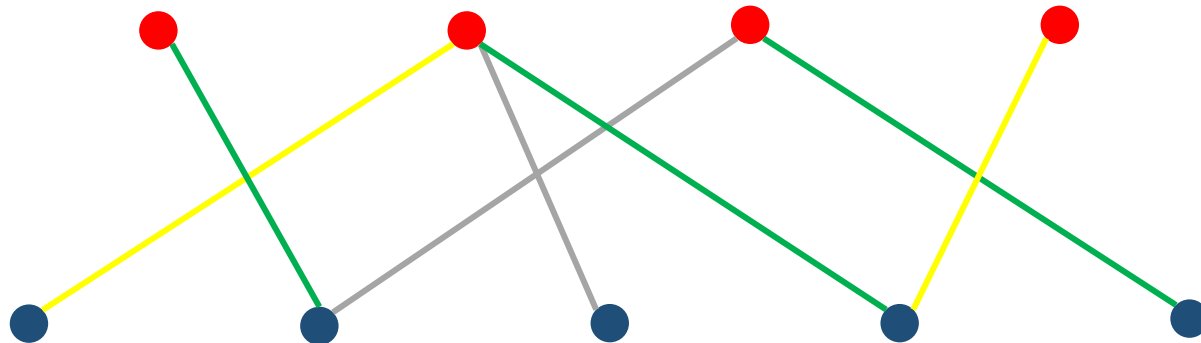
Основные определения

- **Двудольный граф** — граф, множество вершин которого можно разбить на две части так, что каждое ребро соединяет вершины из разных частей.
- **Паросочетание** — множество рёбер без общих вершин.
- **Максимальное паросочетание** — паросочетание максимального размера.



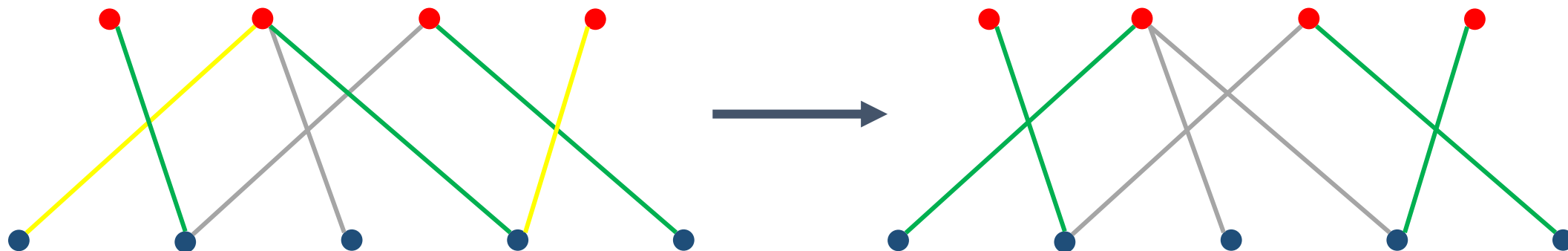
Основные определения

- **Чередующаяся цепь** — путь между двумя вершинами, для которого верно, что среди любых двух соседних рёбер ровно одно принадлежит паросочетанию.
- **Дополняющая цепь** — чередующаяся цепь, оба конца которой не покрыты рёбрами из паросочетания.



Дополняющая цепь: замечание

При помощи дополняющей цепи можно увеличить мощность паросочетания на 1.



Необходимые теоремы

Теорема 1: о дополняющей цепи

Паросочетание максимально тогда и только тогда, когда нет дополняющих цепей.

Теорема 2

Если из вершины v нельзя найти дополняющей цепи, то после дальнейших улучшений текущего паросочетания через дополняющие цепи из v всё ещё не будет дополняющей цепи.

Алгоритм Куна

Из указанных выше теорем получаем следующий алгоритм:

1. Изначально фиксируем пустое паросочетание.
2. Пробегаемся по вершинам и пытаемся при помощи dfs найти дополняющую цепь.
3. Если дополняющая цепь найдена, увеличиваем вдоль неё паросочетание и продолжаем алгоритм.

Алгоритм Куна: замечания

Замечание 1

Если граф двудольный, то достаточно пробежаться лишь по вершинам одной из долей, для определённости — левой.

Замечание 2

В общем случае итоговое время работы $O(nt)$, где n и t — количество вершин и количество рёбер соответственно. В случае же двудольного графа асимптотикой будет $O(n_1t)$, где n_1 — количество вершин в левой доле.

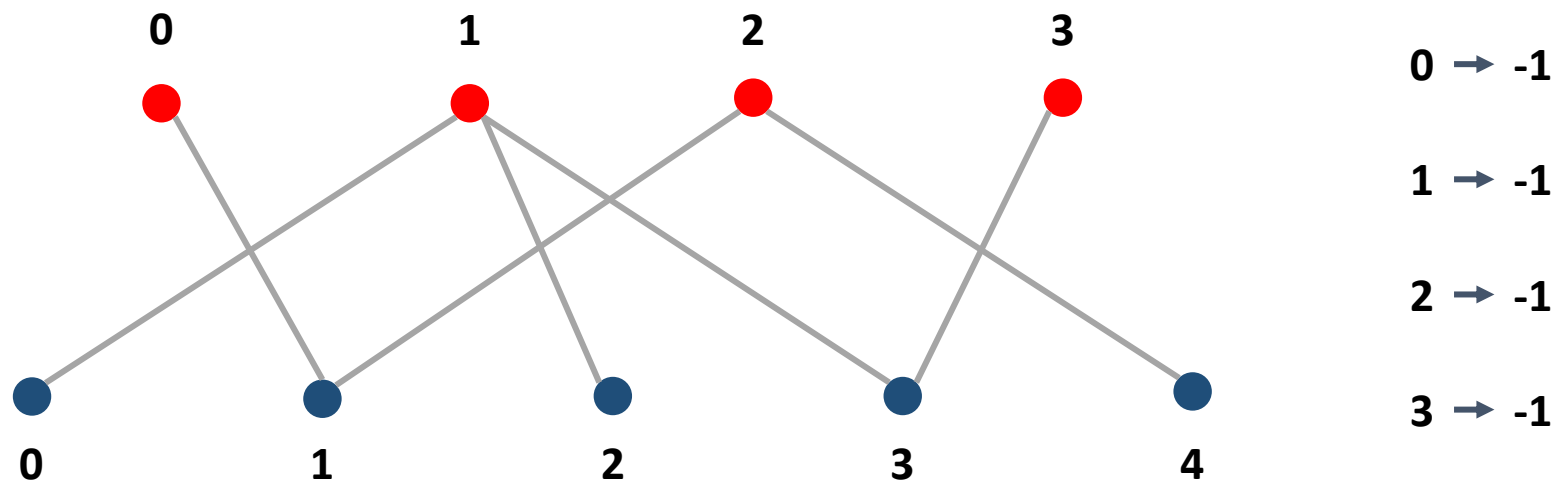
Алгоритм Куна: реализация

Для поиска дополняющей цепи можно завести массив `used`, который будет обнуляться при каждой попытке поиска дополняющей цепи, и массив `matching`, который хранит текущее найденное паросочетание — если вершина лежит на ребре из паросочетания, то ей соответствует в этом массиве номер смежной вершины.

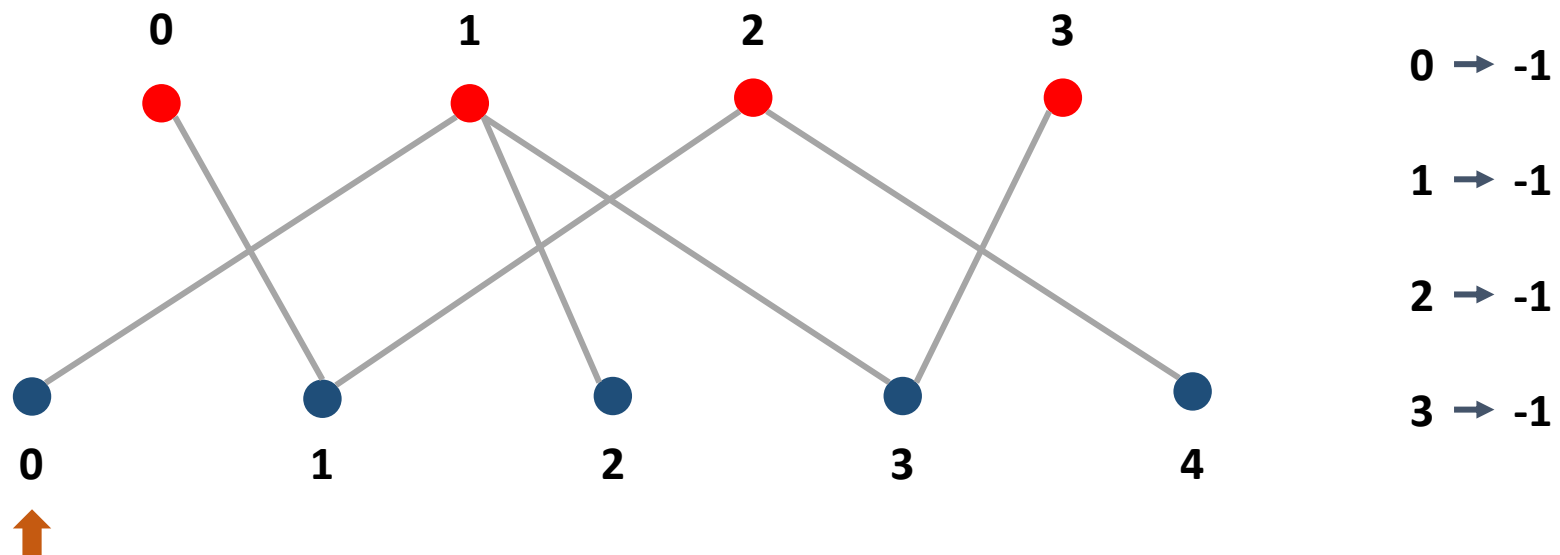
Замечание

Если граф двудольный, то массив `matching` можно завести только под одну долю, а проверку на увеличивающую цепь производить из другой.

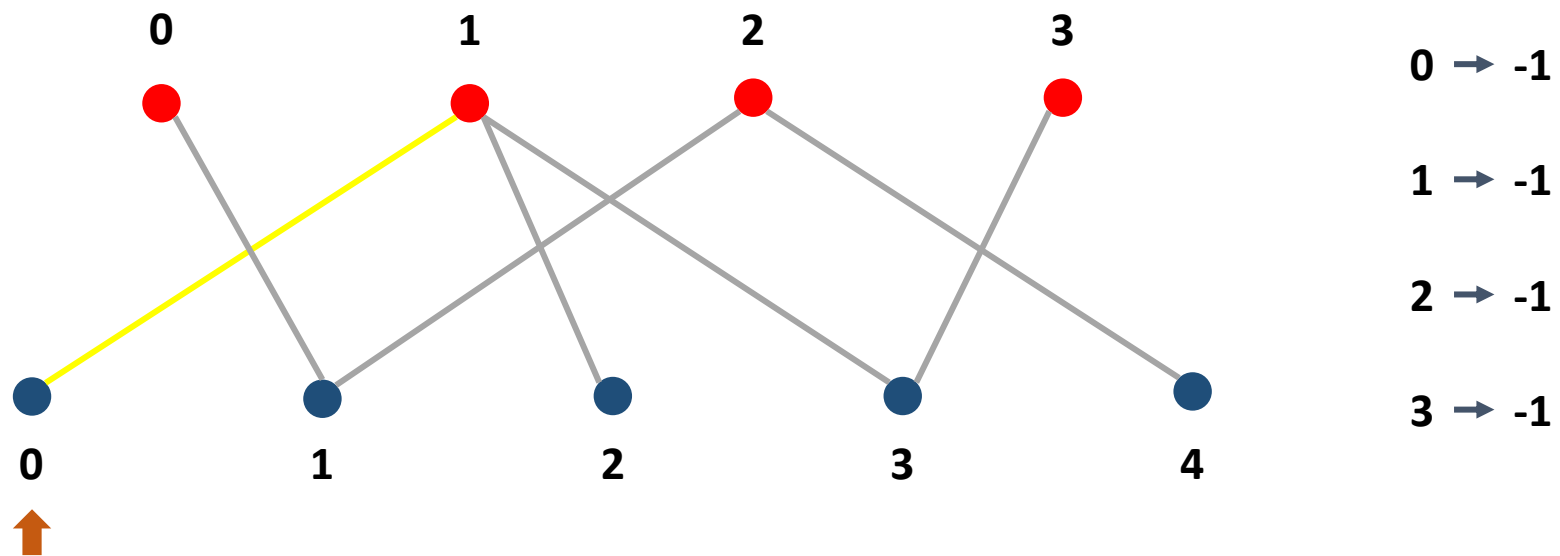
Алгоритм Куна: пример



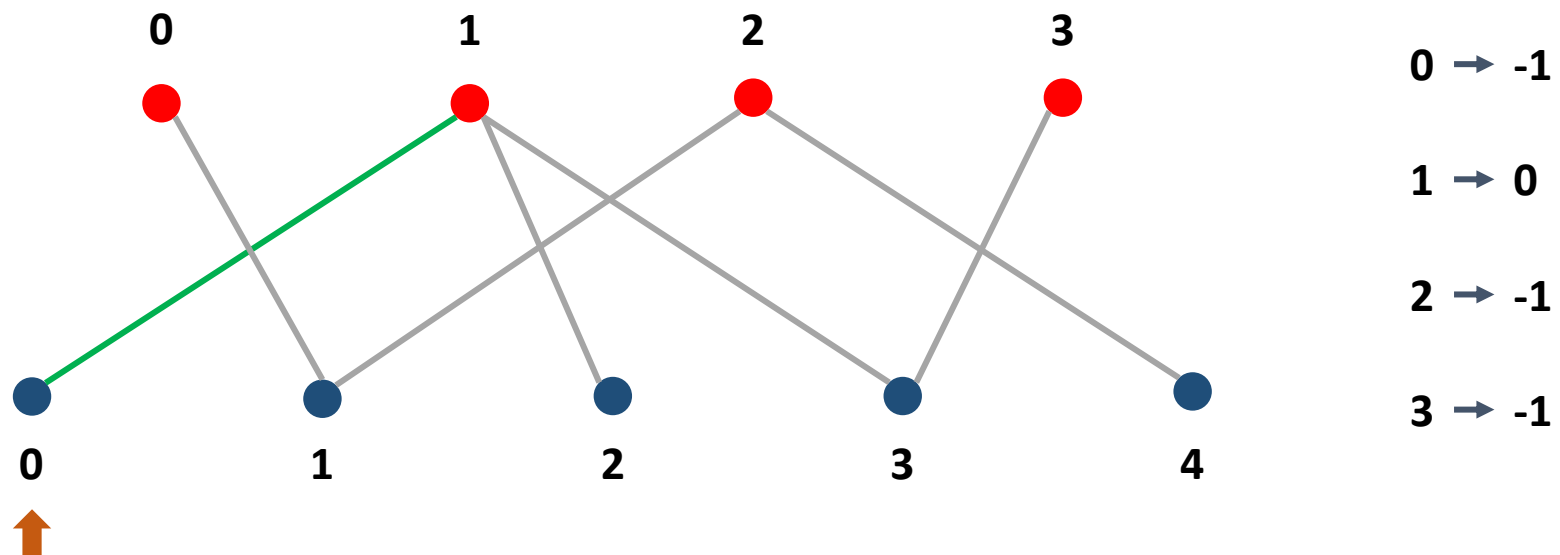
Алгоритм Куна: пример



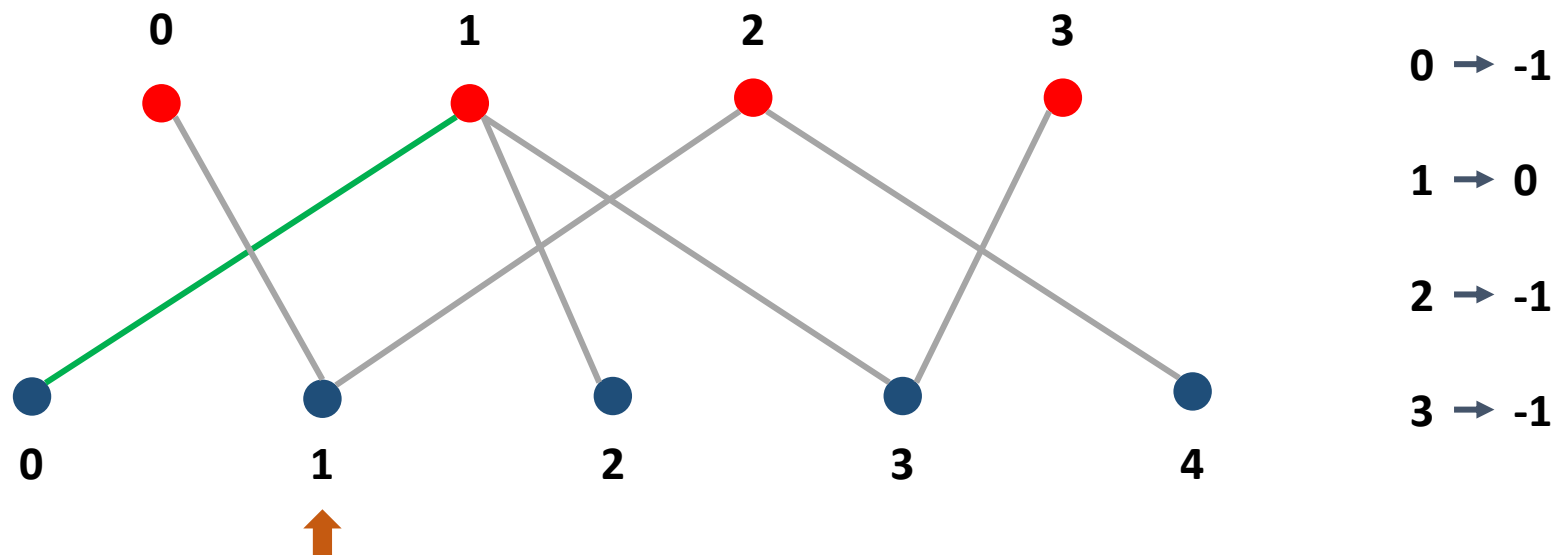
Алгоритм Куна: пример



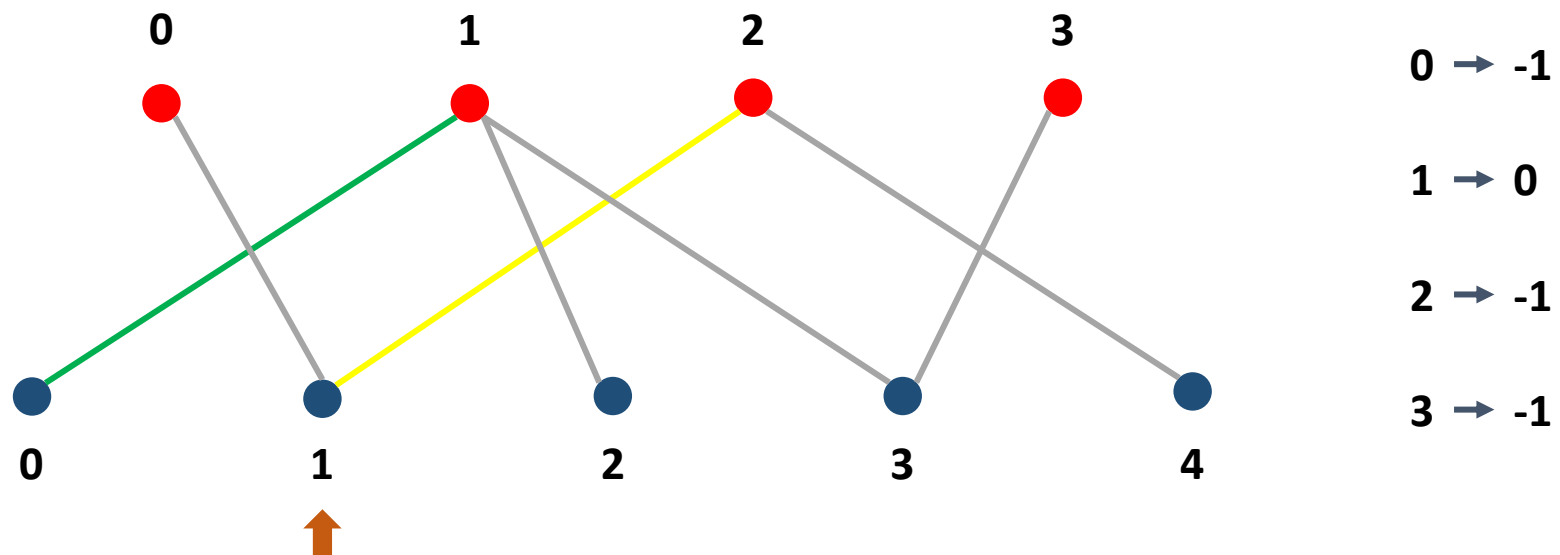
Алгоритм Куна: пример



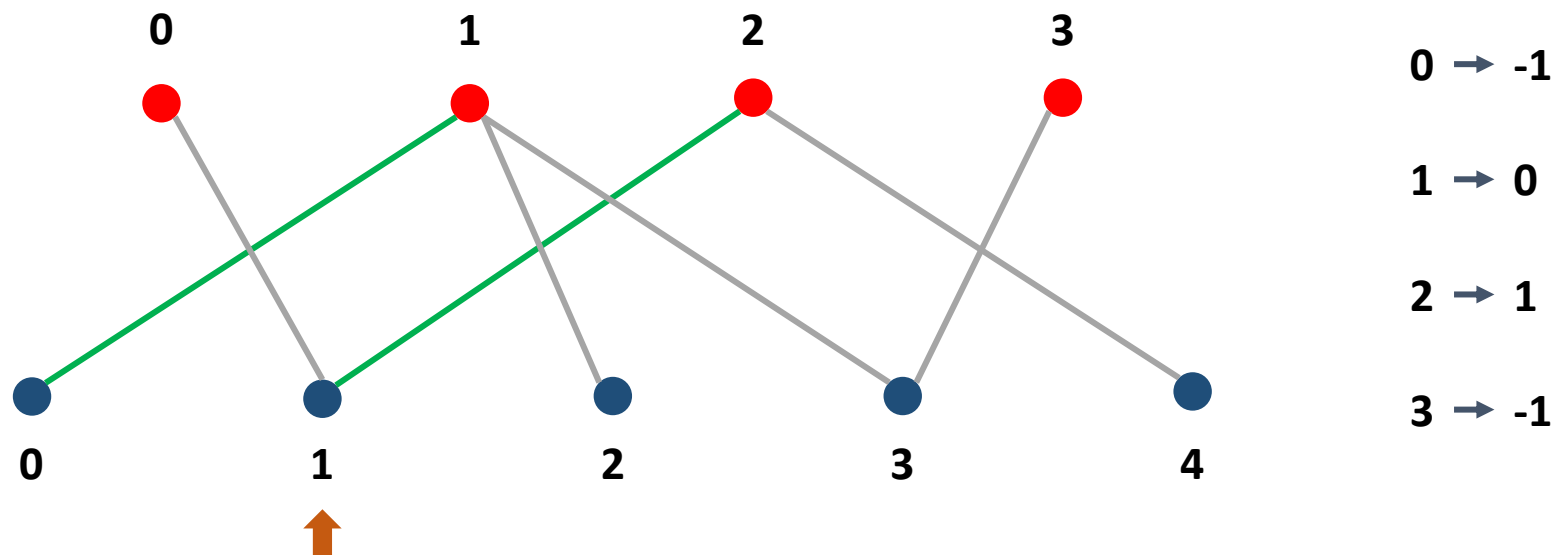
Алгоритм Куна: пример



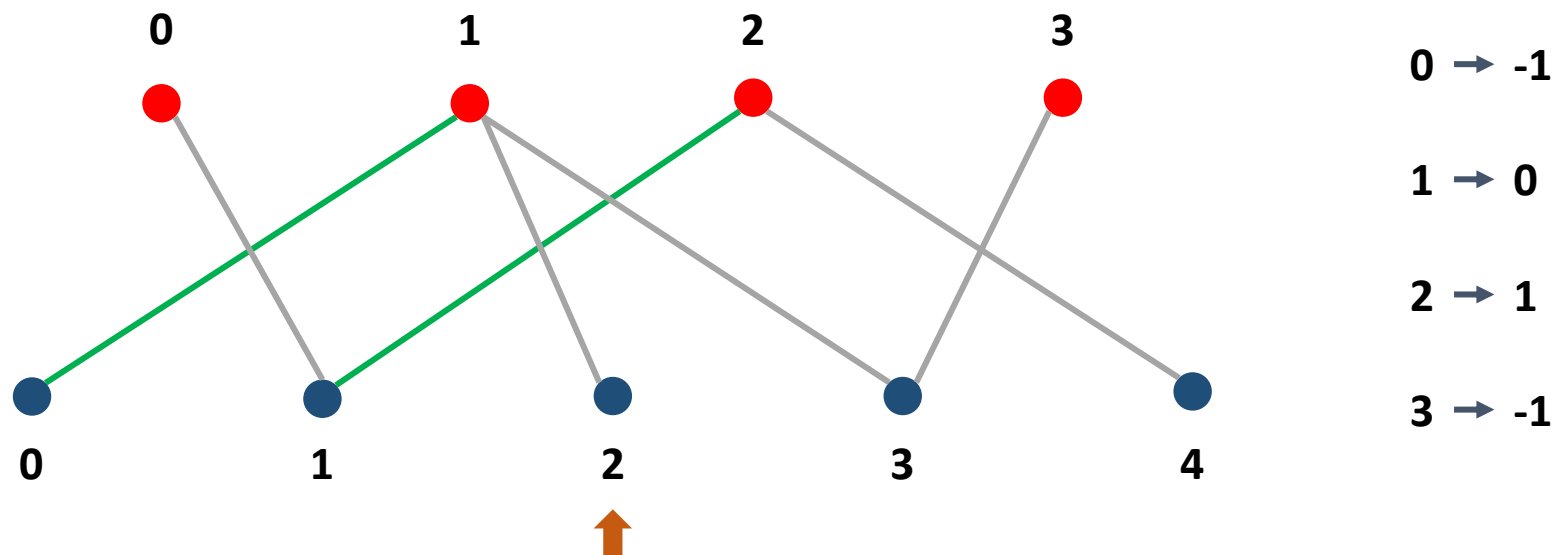
Алгоритм Куна: пример



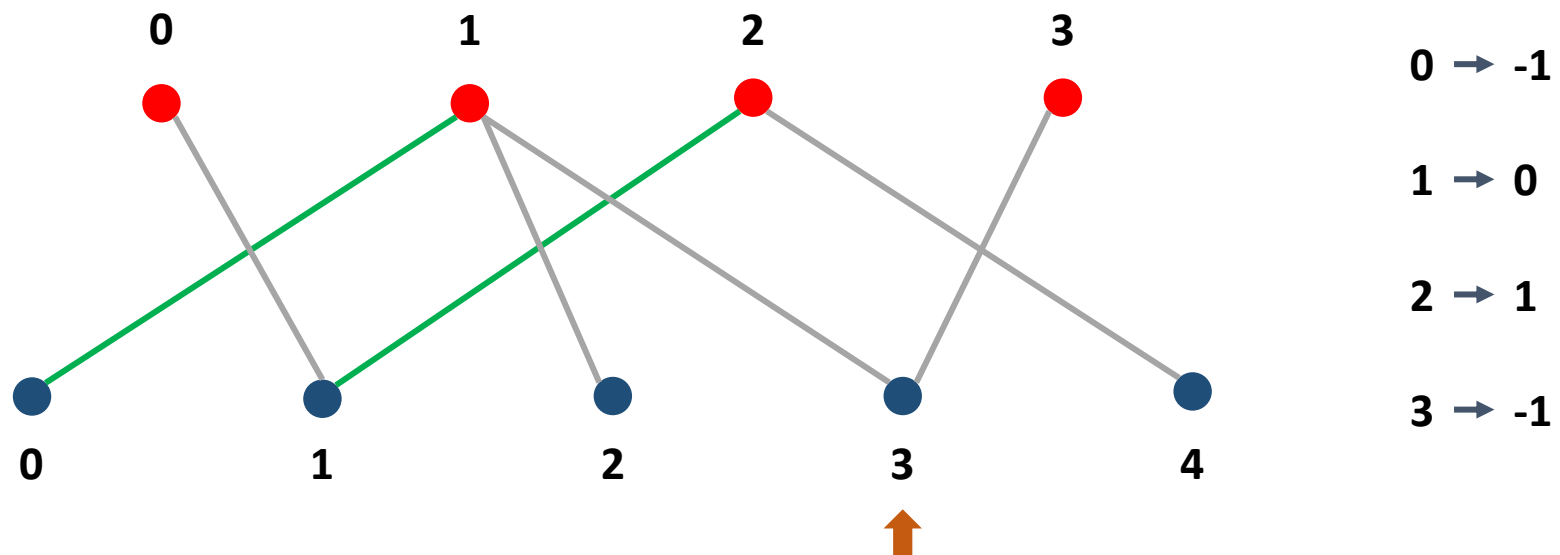
Алгоритм Куна: пример



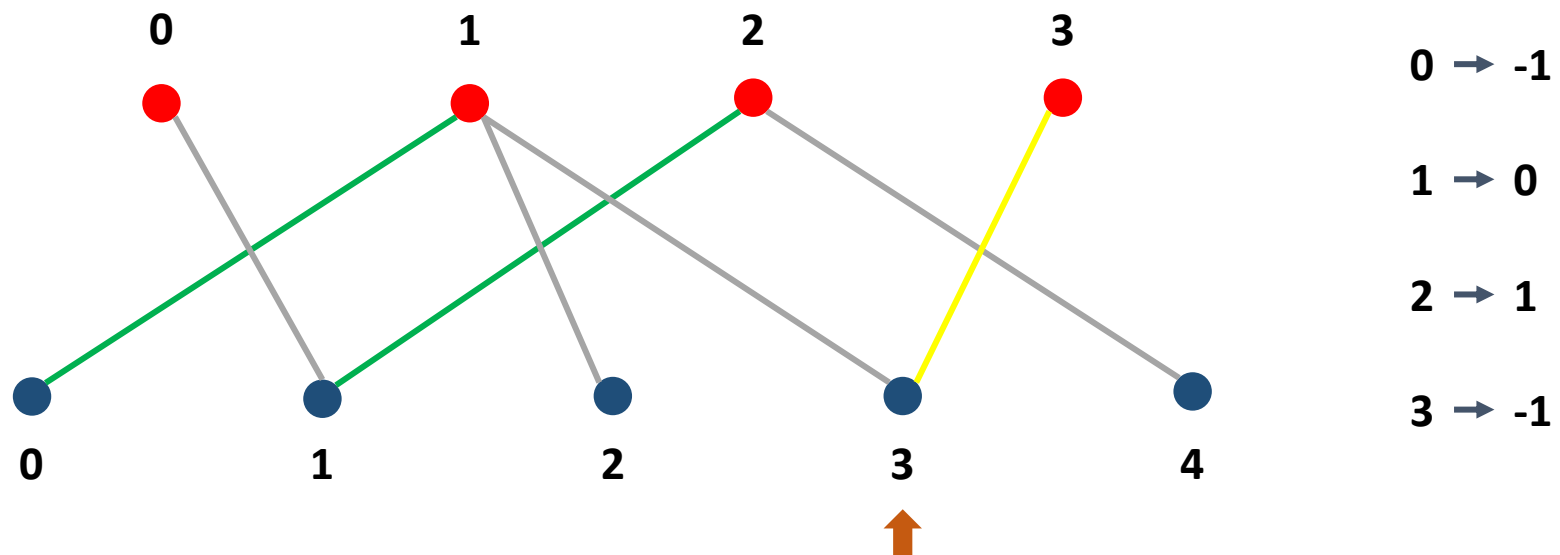
Алгоритм Куна: пример



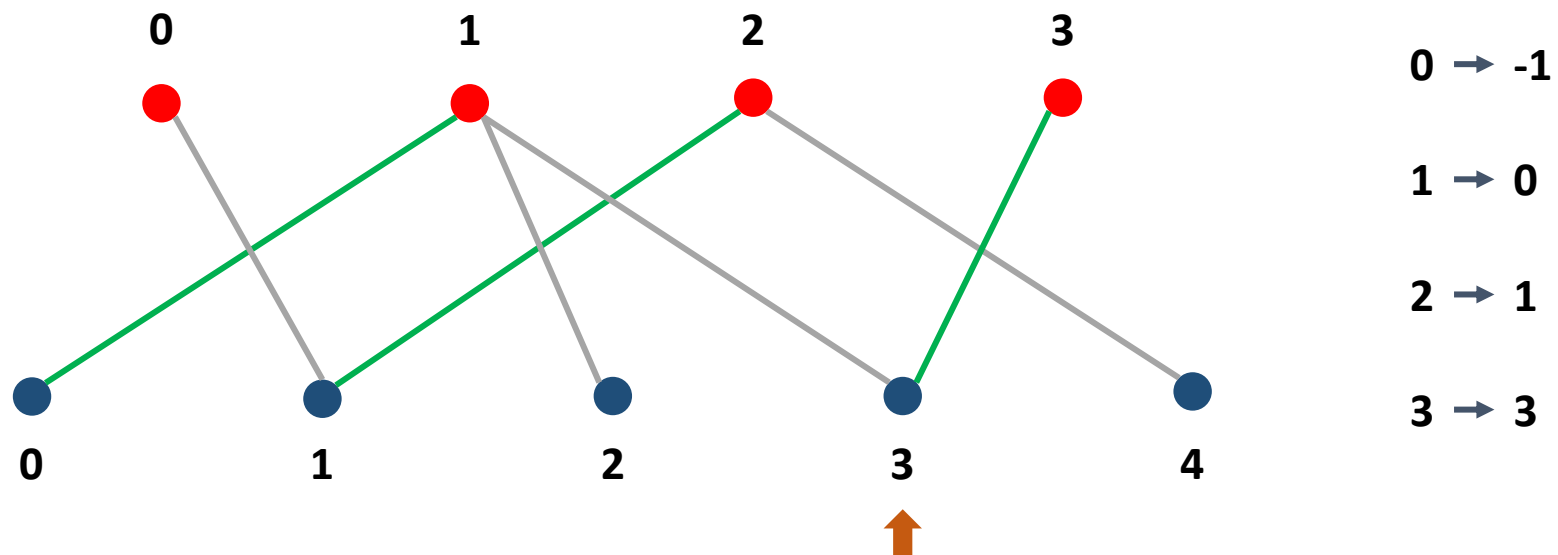
Алгоритм Куна: пример



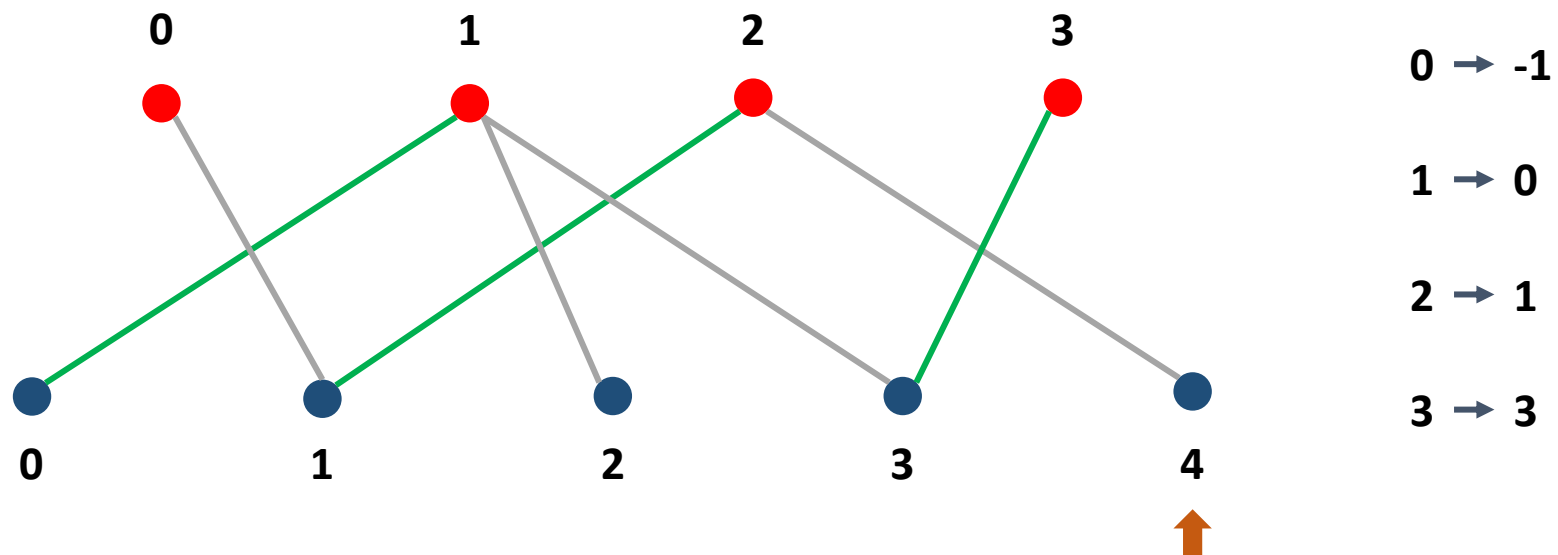
Алгоритм Куна: пример



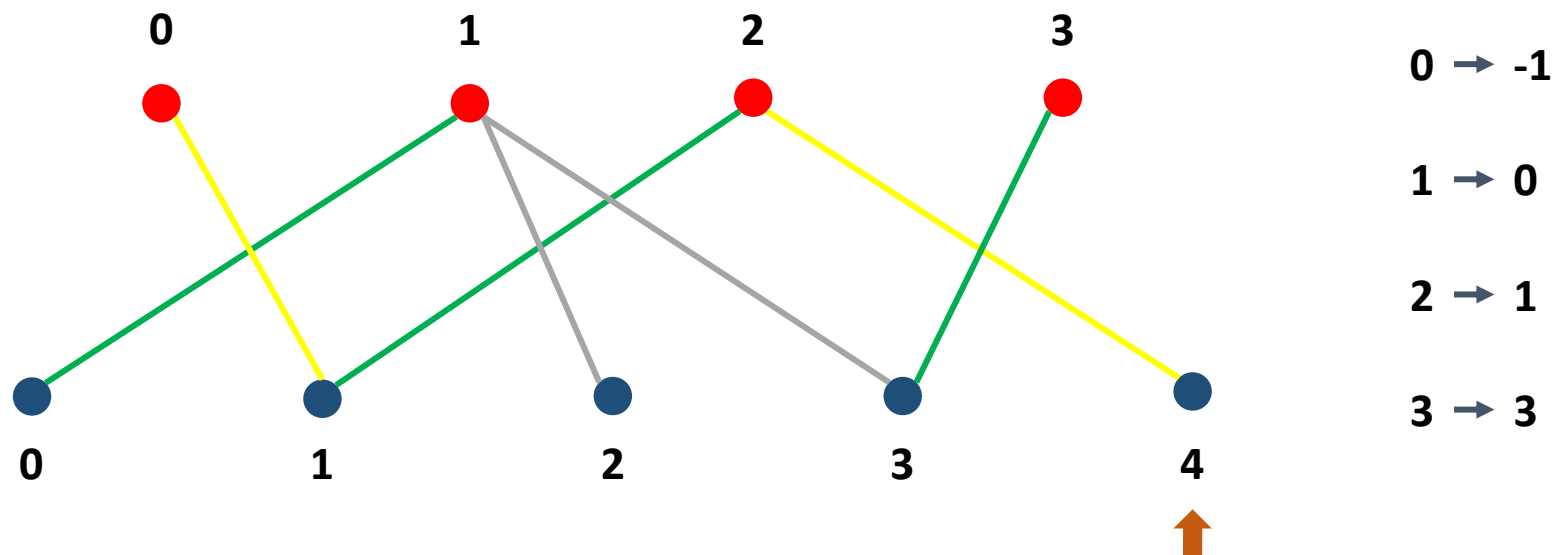
Алгоритм Куна: пример



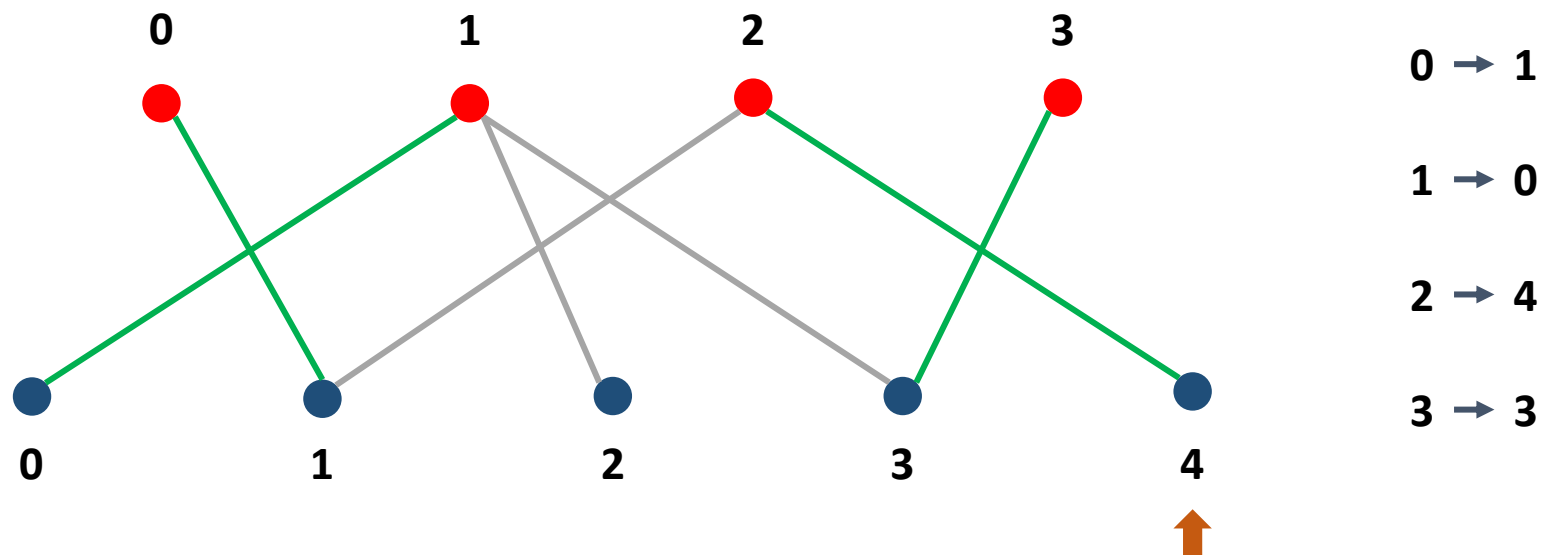
Алгоритм Куна: пример



Алгоритм Куна: пример

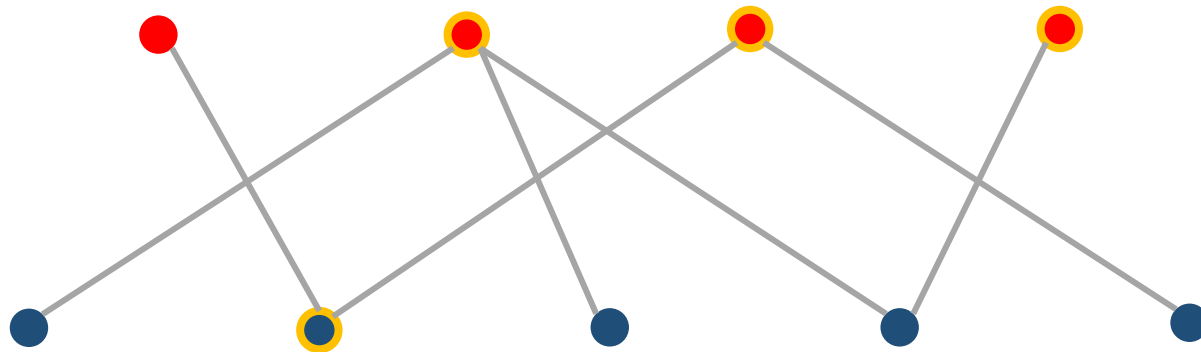


Алгоритм Куна: пример



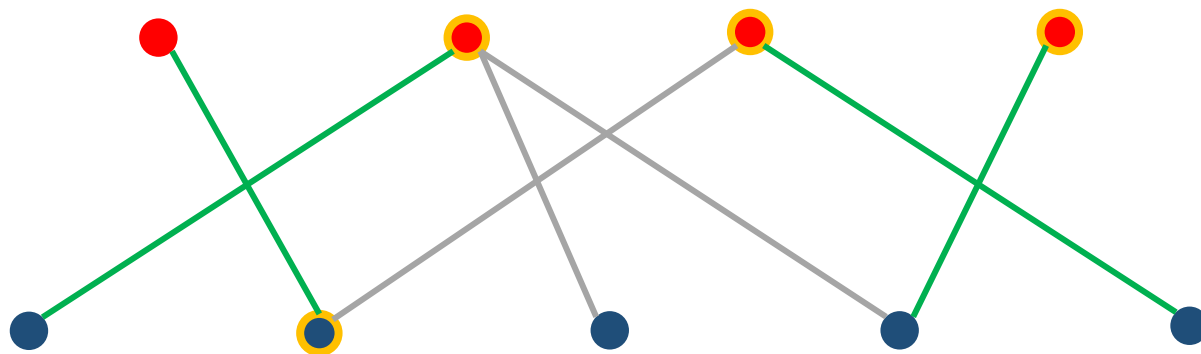
Вершинное покрытие

- Вершинным покрытием называется множество вершин графа такое, что каждое ребро содержит вершину из этого множества.
- Мы хотим найти минимальное вершинное покрытие.



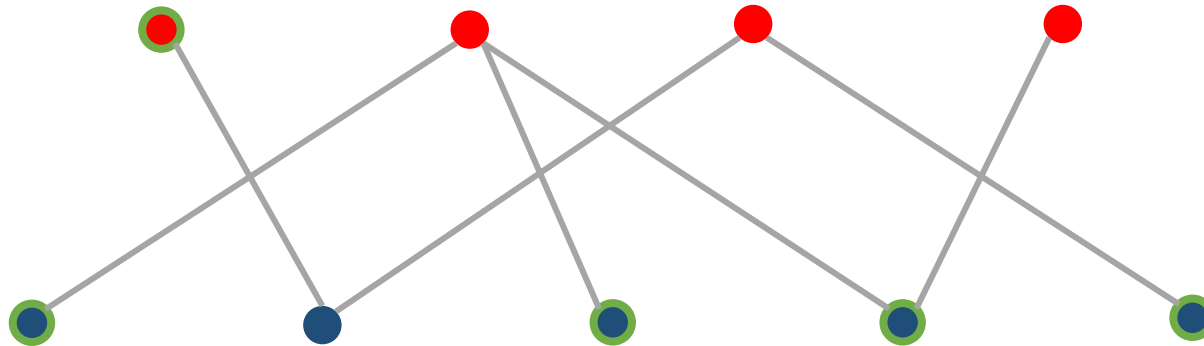
Теорема Кёнига

В двудольном графе мощности максимального паросочетания и минимального вершинного покрытия совпадают.



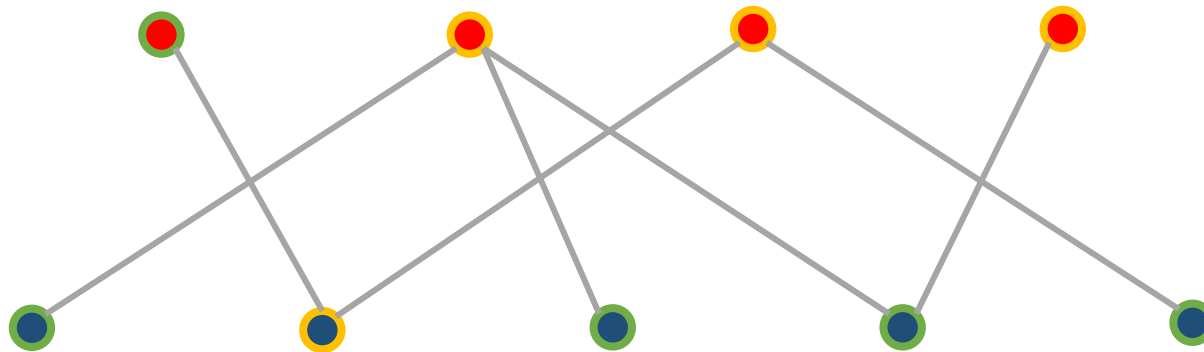
Независимое множество

- Независимым множеством называется множество вершин графа такое, что никакие две вершины из этого множества не соединены ребром.
- Мы хотим найти максимальное независимое множество.



Связь вершинного покрытия и независимого множества

- Дополнение вершинного покрытия — независимое множество.
- Дополнение независимого множества — вершинное покрытие.
- $|МНМ| = |V| - |МВП|$



Венгерский алгоритм: дополнительные определения

- **Полный двудольный граф** — граф, между каждой левой и каждой правой вершиной которого существует ребро.
- **Полное паросочетание** — паросочетание, которое покрывает все вершины.

Задача о назначениях

- Дан взвешенный полный двудольный граф, каждое ребро которого имеет неотрицательный вес. Необходимо найти в нём полное паросочетание минимального веса.
- Другая формулировка задачи: есть n работ и n исполнителей. Для каждой пары исполнитель-работа известна стоимость, с которой исполнитель может быть нанят на работу. Необходимо на каждую работу найти по исполнителю с минимальными суммарными затратами.

Венгерский алгоритм

- Имеем матрицу $a[1..n][1..n]$.
- $a[i][j] \geq 0$
- $u[1..n]$ и $v[1..n]$ — потенциалы.
- $u[i] + v[j] \leq a[i][j]$
- $f = \sum_{i=1}^n u[i] + \sum_{j=1}^n v[j]$
- Если $u[i] + v[j] = a[i][j]$, то ребро (i, j) называется жестким.

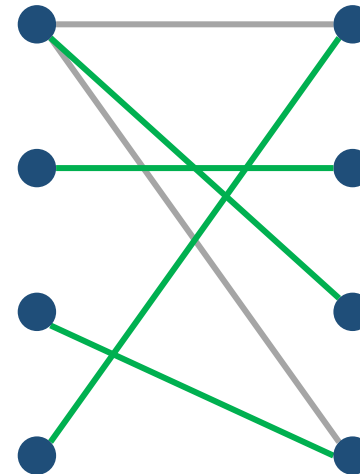
Венгерский алгоритм

- Пусть S — стоимость искомого решения. Тогда для любого потенциала верно, что $S \geq f$. Это следует из того, что $u[i] + v[j] \leq a[i][j]$. Так как все элементы находятся в разных строках и столбцах, то, суммируя их, получаем требуемое неравенство.
- Если какое-либо решение имеет стоимость, равную по величине какому-либо потенциалу, то это решение оптимально.
- Всегда существует решение и потенциал, на которых неравенство обращается в равенство (будет следовать из алгоритма).

Венгерский алгоритм

Пусть H — двудольный граф, составленный только из жестких рёбер. Суть Венгерского алгоритма в том, что он поддерживает максимальное по количеству рёбер паросочетание M в графе H , и как только оно станет содержать n рёбер, рёбра этого паросочетания будут являться ответом.

		v			
		-7	0	0	-2
u	9	2	10	9	7
	4	15	4	14	8
	13	13	14	16	11
	11	4	15	13	19



Венгерский алгоритм

Непосредственно алгоритм:

- В начале алгоритма $u[i] = v[i] = 0$, паросочетание M пустое.
- Не меняя потенциала, пытаемся увеличить мощность паросочетания на единицу. Если этого не удалось сделать, то в случае с максимальным паросочетанием ответ найден, в противном случае переходим к следующему шагу.
- Мощность паросочетания находится подобно алгоритму Куна: из всех ненасыщенных вершин левой доли запускается обход в глубин/ширину, который идёт только по чередующимся цепочкам. Если в процессе обхода удалось посетить ненасыщенную вершину правой доли, то найдена увеличивающая цепь, вдоль которой увеличиваем паросочетание.

Венгерский алгоритм

- Пересчитываем потенциал таким образом, чтобы на следующих шагах было больше возможностей для увеличения паросочетания.
- Z_1 и Z_2 — множества вершин левой и правой долей соответственно, которые были посещены в процессе предыдущего шага.
- $\delta = \min_{i \in Z_1, j \notin Z_2} \{a[i][j] - u[i] - v[j]\}$
- $\delta > 0$
- Пересчитываем потенциал: $u[i] += \delta \quad \forall i \in Z_1, \quad v[j] -= \delta \quad \forall j \in Z_2$
- Полученный потенциал корректен.
- Все рёбра паросочетания останутся жёсткими.
- $|Z_1| + |Z_2|$ строго увеличивается, то есть будет конечное число изменений потенциала.
- Произойдёт не более n пересчётов потенциала, прежде чем найдётся увеличивающая цепочка.

Венгерский алгоритм: время работы

Всего должно произойти n увеличений паросочетания, каждое увеличение требует не более n пересчётов потенциала, каждый потенциал пересчитывается за $O(n^2)$. Итоговая асимптотика: $O(n^4)$.

Венгерский алгоритм: ускорение

Ключевая идея ускоренного алгоритма: добавляем в рассмотрение строки матрицы одна за одной, а не рассматриваем их все сразу.
Непосредственно алгоритм:

- Добавляем в рассмотрение очередную строку матрицы.
- Пока нет увеличивающей цепи, начинающейся в этой строке, пересчитываем потенциал.
- При появлении увеличивающей цепи, чередуем вдоль неё паросочетание и переходим к следующей строке.

Венгерский алгоритм: ускорение

Хотим быстро выполнять последние два шага. Учтём следующие два факта:

- При изменении потенциала вершины, которые были достижимы обходом Куна, останутся достижимыми.
- Всего произойдёт $O(n)$ пересчётов потенциала, прежде чем будет найдена увеличивающая цепь.

Венгерский алгоритм: ускорение

Из фактов выше вытекают две идеи:

- Алгоритм поиска увеличивающей цепи можно выполнять итеративно: после каждого пересчёта потенциала мы просматриваем добавившиеся жёсткие рёбра и, если их левые концы были достижимыми, помечаем их правые концы также достижимыми и продолжаем обход из них.
- Получаем цикл, на каждом шаге которого сначала пересчитывается потенциал, затем находится столбец, ставший достижимым (такой всегда найдётся), и если этот столбец был ненасыщен, то найдена увеличивающая цепь, а если столбец был насыщен — то соответствующая ему в паросочетании строка также становится достижимой.

В итоге второй и третий шаги алгоритма сводятся к циклу добавления столбцов, на каждой итерации которого сначала пересчитывается потенциал, а затем какой-то новый столбец помечается как достижимый.

Венгерский алгоритм: ускорение

Осталось научиться быстро пересчитывать потенциал. Для этого будем поддерживать вспомогательные минимумы по каждому из столбцов:

- $minv[j] = \min_{i \in Z_1} \{a[i][j] - u[i] - v[j]\}$
- $\delta = \min_{j \notin Z_2} \{minv[j]\}$

Массив изменяется при добавлении новых строк и при пересчёте потенциала. В каждом случае пересчёт идёт за $O(n)$.

Венгерский алгоритм: время работы

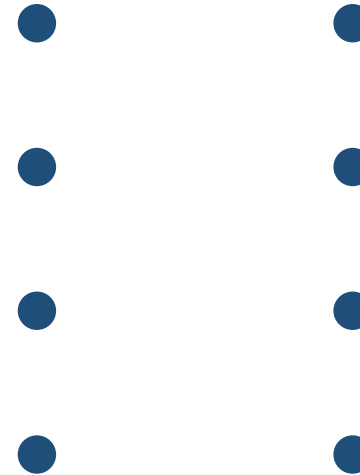
Всего будет добавлено в рассмотрение n строк. Обработка очередной строки включает в себя:

- $O(n)$ пересчётов потенциала, каждый из которых будет занимать $O(n)$ времени на поддержание массива minv .
- Поиск увеличивающей цепи за $O(n^2)$.

Итоговая асимптотика: $O(n^3)$.

Венгерский алгоритм: пример

		v			
		0	0	0	0
u	0	2	10	9	7
	0	15	4	14	8
	0	13	14	16	11
	0	4	15	13	19



Венгерский алгоритм: пример

		v			
		0	0	0	0
u	0	2	10	9	7
	0	15	4	14	8
	0	13	14	16	11
	0	4	15	13	19



Венгерский алгоритм: пример

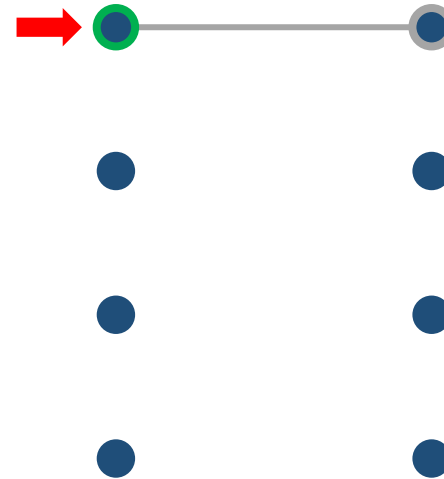
		v			
		0	0	0	0
u	0	2	10	9	7
	0	15	4	14	8
	0	13	14	16	11
	0	4	15	13	19

$\delta = 2$



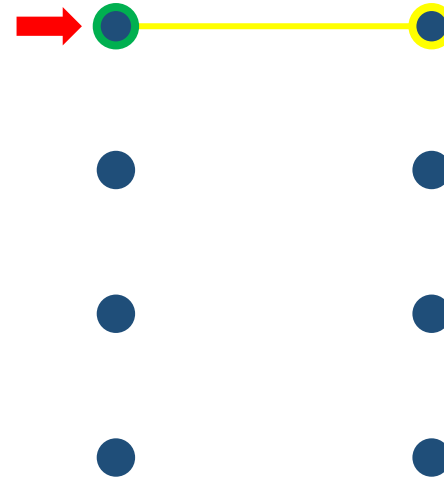
Венгерский алгоритм: пример

		v				
		0	0	0	0	
u	2	2	10	9	7	+2
	0	15	4	14	8	
	0	13	14	16	11	
	0	4	15	13	19	



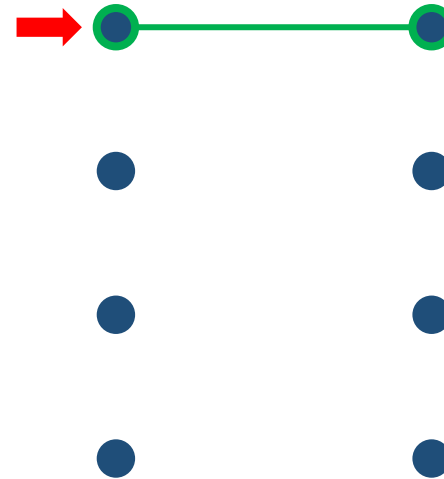
Венгерский алгоритм: пример

		v			
		0	0	0	0
u	2	2	10	9	7
	0	15	4	14	8
	0	13	14	16	11
	0	4	15	13	19



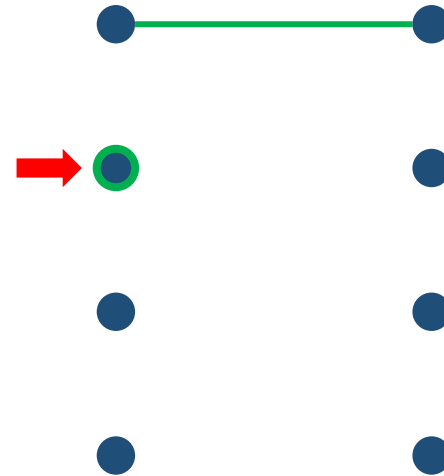
Венгерский алгоритм: пример

		v			
		0	0	0	0
u	2	2	10	9	7
	0	15	4	14	8
	0	13	14	16	11
	0	4	15	13	19



Венгерский алгоритм: пример

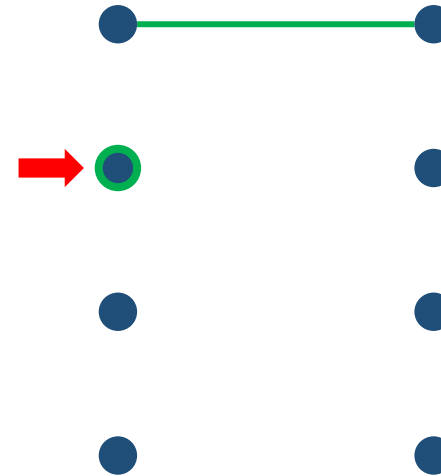
		v			
		0	0	0	0
u	2	2	10	9	7
	0	15	4	14	8
	0	13	14	16	11
	0	4	15	13	19



Венгерский алгоритм: пример

		v			
		0	0	0	0
u	2	2	10	9	7
	0	15	4	14	8
	0	13	14	16	11
	0	4	15	13	19

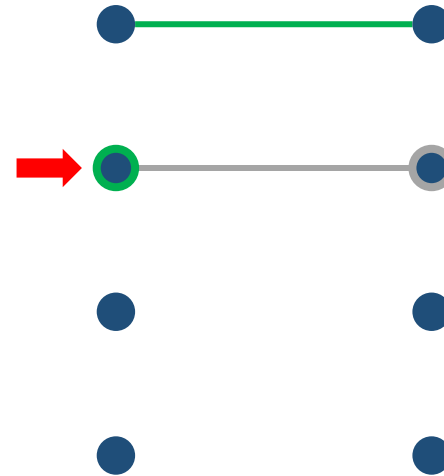
$\delta = 4$



Венгерский алгоритм: пример

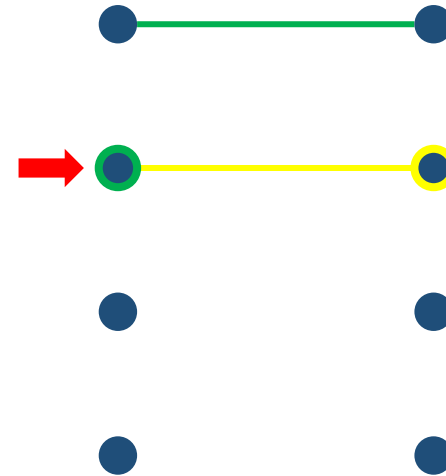
		v			
		0	0	0	0
u	2	2	10	9	7
	4	15	4	14	8
	0	13	14	16	11
	0	4	15	13	19

+4



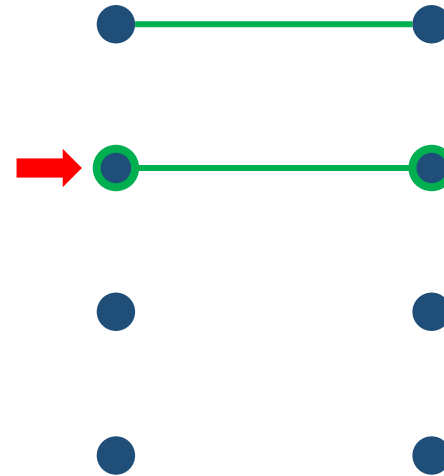
Венгерский алгоритм: пример

		v			
		0	0	0	0
u	2	2	10	9	7
	4	15	4	14	8
	0	13	14	16	11
	0	4	15	13	19



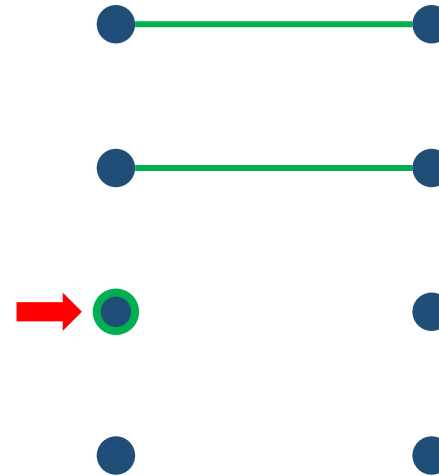
Венгерский алгоритм: пример

		v			
		0	0	0	0
u	2	2	10	9	7
	4	15	4	14	8
	0	13	14	16	11
	0	4	15	13	19



Венгерский алгоритм: пример

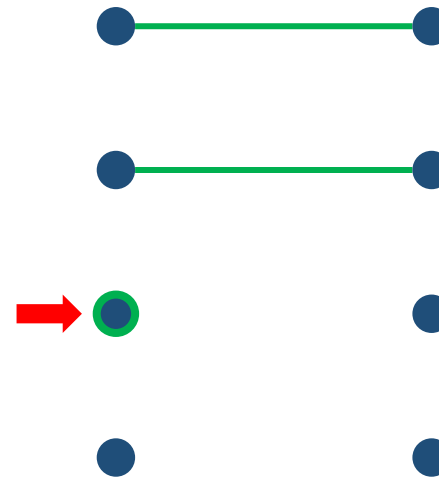
		v			
		0	0	0	0
u	2	2	10	9	7
	4	15	4	14	8
	0	13	14	16	11
	0	4	15	13	19



Венгерский алгоритм: пример

		v			
		0	0	0	0
u	2	2	10	9	7
	4	15	4	14	8
	0	13	14	16	11
	0	4	15	13	19

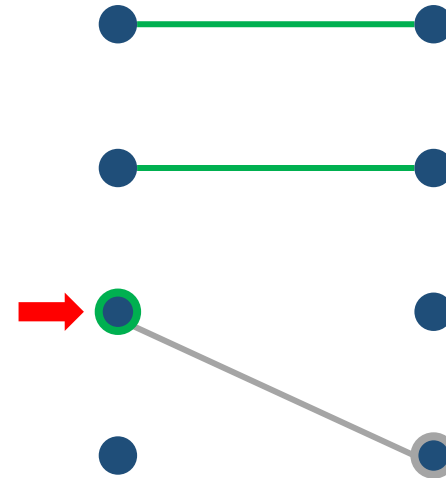
$\delta = 11$



Венгерский алгоритм: пример

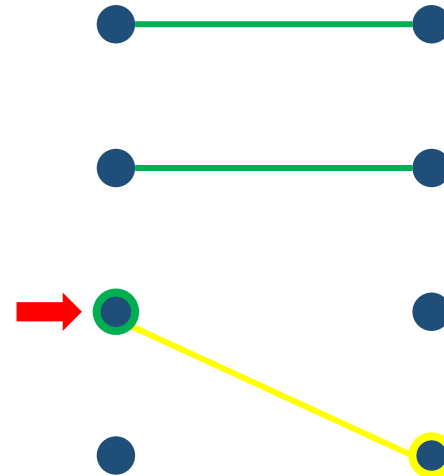
		v			
		0	0	0	0
u	2	2	10	9	7
	4	15	4	14	8
	11	13	14	16	11
	0	4	15	13	19

+11



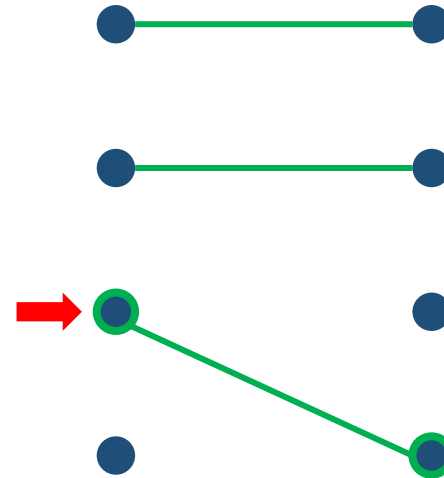
Венгерский алгоритм: пример

		v			
		0	0	0	0
u	2	2	10	9	7
	4	15	4	14	8
	11	13	14	16	11
	0	4	15	13	19



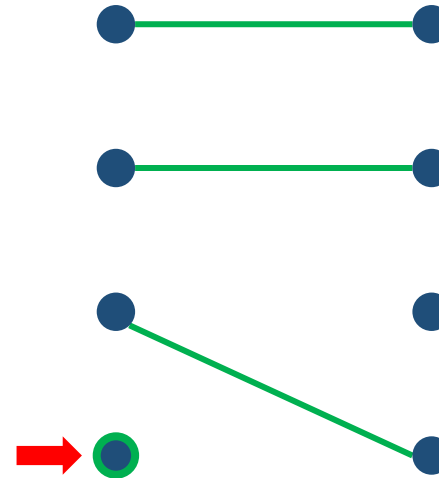
Венгерский алгоритм: пример

		v			
		0	0	0	0
u	2	2	10	9	7
	4	15	4	14	8
	11	13	14	16	11
	0	4	15	13	19



Венгерский алгоритм: пример

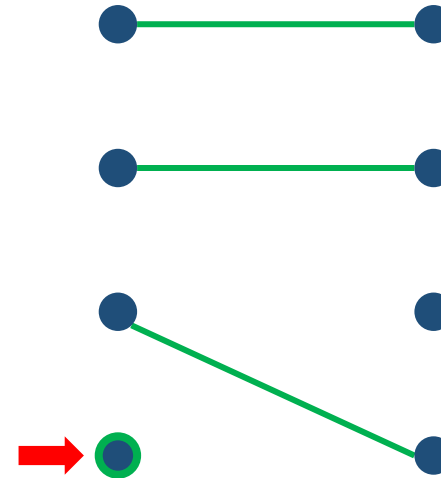
		v			
		0	0	0	0
u	2	2	10	9	7
	4	15	4	14	8
	11	13	14	16	11
	0	4	15	13	19



Венгерский алгоритм: пример

		v			
		0	0	0	0
u	2	2	10	9	7
	4	15	4	14	8
	11	13	14	16	11
	0	4	15	13	19

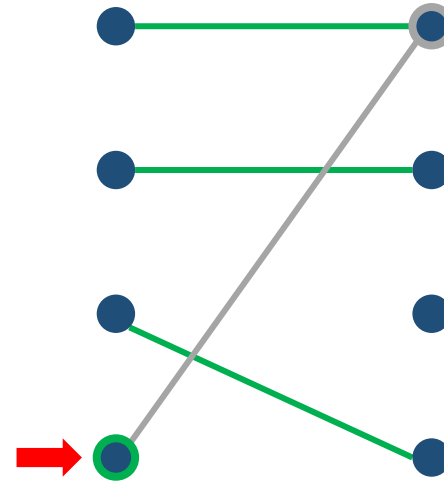
$\delta = 4$



Венгерский алгоритм: пример

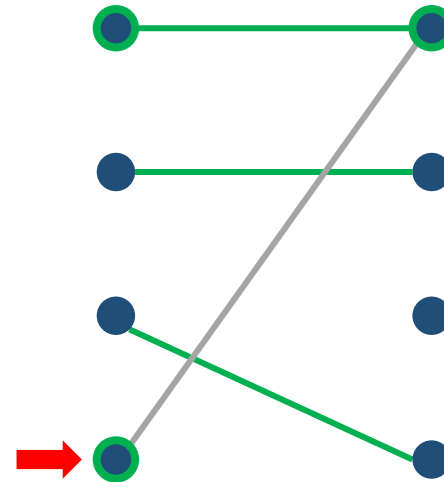
		v			
		0	0	0	0
u	2	2	10	9	7
	4	15	4	14	8
	11	13	14	16	11
	4	4	15	13	19

+4



Венгерский алгоритм: пример

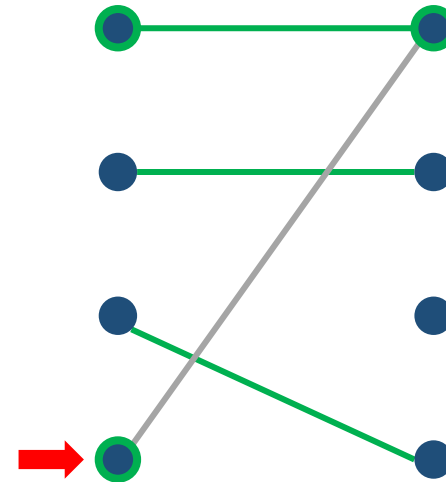
		v			
		0	0	0	0
u	2	2	10	9	7
	4	15	4	14	8
	11	13	14	16	11
	4	4	15	13	19



Венгерский алгоритм: пример

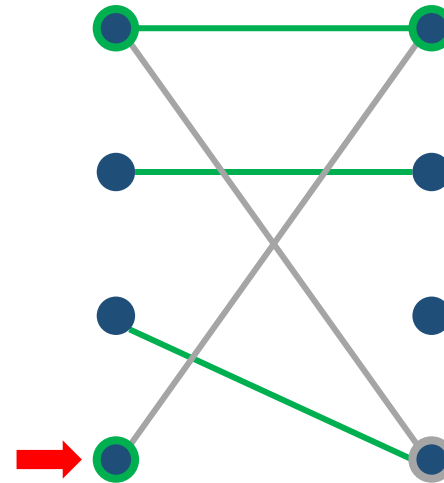
		v			
		0	0	0	0
u	2	2	10	9	7
	4	15	4	14	8
	11	13	14	16	11
	4	4	15	13	19

$\delta = 5$



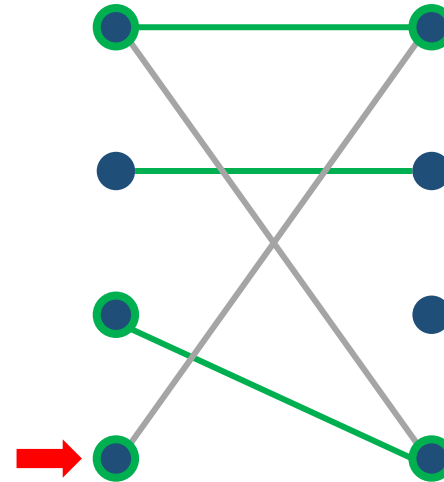
Венгерский алгоритм: пример

		v				
		-5	0	0	0	
u	7	2	10	9	7	+5
	4	15	4	14	8	
	11	13	14	16	11	
	9	4	15	13	19	+5
		-5				



Венгерский алгоритм: пример

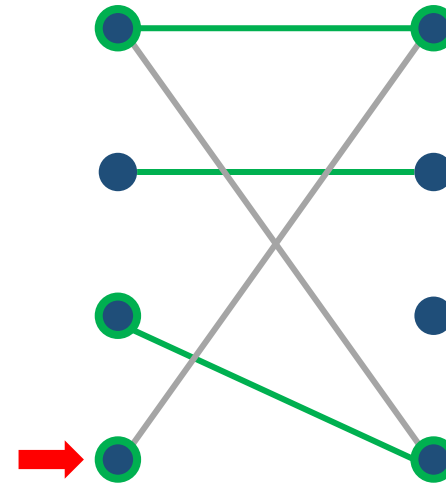
		v			
		-5	0	0	0
u	7	2	10	9	7
	4	15	4	14	8
	11	13	14	16	11
	9	4	15	13	19



Венгерский алгоритм: пример

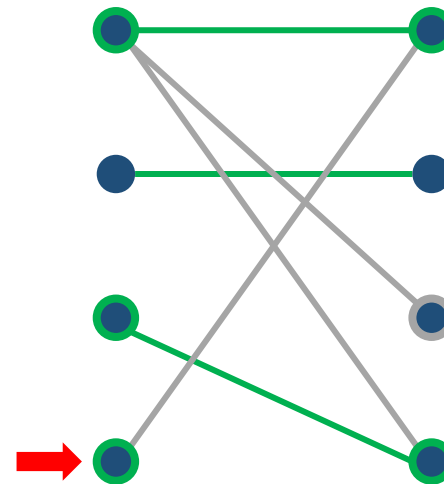
		v			
		-5	0	0	0
u	7	2	10	9	7
	4	15	4	14	8
	11	13	14	16	11
	9	4	15	13	19

$\delta = 2$



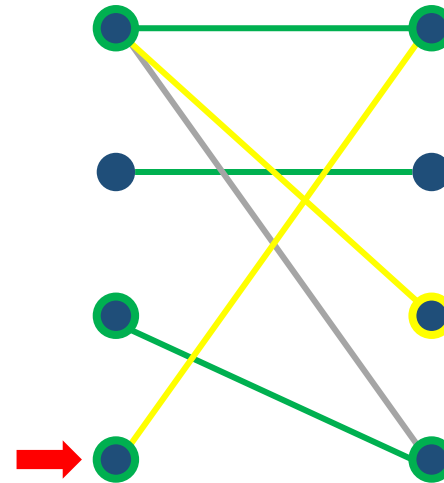
Венгерский алгоритм: пример

		v				
		-7	0	0	-2	
u	9	2	10	9	7	+2
	4	15	4	14	8	
	13	13	14	16	11	+2
	11	4	15	13	19	+2
		-2		-2		



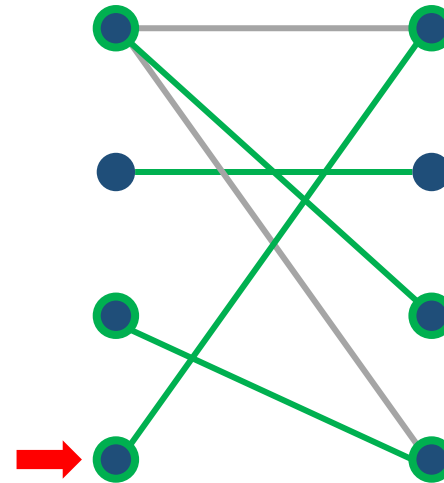
Венгерский алгоритм: пример

		v			
		-7	0	0	-2
u	9	2	10	9	7
	4	15	4	14	8
	13	13	14	16	11
	11	4	15	13	19



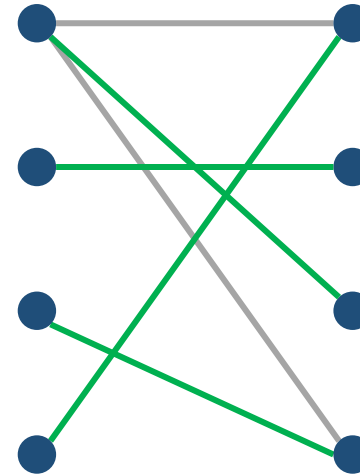
Венгерский алгоритм: пример

		v			
		-7	0	0	-2
u	9	2	10	9	7
	4	15	4	14	8
	13	13	14	16	11
	11	4	15	13	19



Венгерский алгоритм: пример

		v			
		-7	0	0	-2
u	9	2	10	9	7
	4	15	4	14	8
	13	13	14	16	11
	11	4	15	13	19



Stop

Дальше идёт решение задачи о назначениях за $O(n^5)$ с нуля. Оно не нужно. На этом стоит остановиться.

Венгерский алгоритм: дополнительные определения

- **Полный двудольный граф** — граф, между каждой левой и каждой правой вершиной которого существует ребро.
- **Полное паросочетание** — паросочетание, которое покрывает все вершины.

Задача о назначениях

- Дан взвешенный полный двудольный граф, каждое ребро которого имеет неотрицательный вес. Необходимо найти в нём полное паросочетание минимального веса.
- Другая формулировка задачи: есть n работ и n исполнителей. Для каждой пары исполнитель-работа известна стоимость, с которой исполнитель может быть нанят на работу. Необходимо на каждую работу найти по исполнителю с минимальными суммарными затратами.

Необходимые леммы

Лемма 1

Если веса всех ребер графа, инцидентных какой-либо вершине, изменить (увеличить или уменьшить) на одно и то же число, то в новом графе оптимальное паросочетание будет состоять из тех же ребер, что и в старом.

Лемма 2

Если веса всех ребер графа неотрицательны и некоторое полное паросочетание состоит из ребер нулевого веса, то оно является оптимальным.

Необходимые леммы

Лемма 3

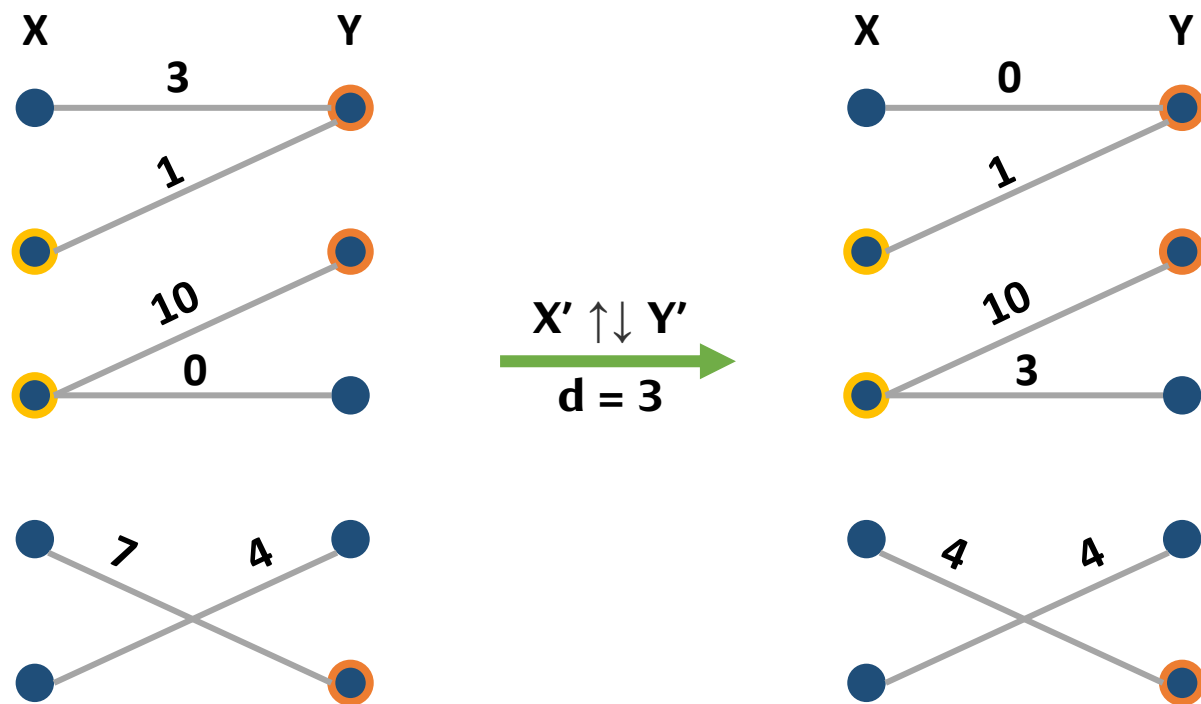
Выделим в множествах X и Y подмножества X' и Y' .

Пусть $d = \min\{c(x, y) \mid x \in X \setminus X', y \in Y'\}$, где $c(x, y)$ — вес ребра (x, y) .

Прибавим d ко всем весам ребер, инцидентных вершинам из X' . Затем отнимем d от всех весов ребер, инцидентных вершинам из Y' (далее для краткости эта операция обозначается за $X' \uparrow \downarrow Y'$). Тогда:

1. Веса всех рёбер графа останутся неотрицательными.
2. Веса рёбер вида (x, y) , где $x \in X'$, $y \in Y'$ или $x \in X \setminus X'$, $y \in Y \setminus Y'$, не изменятся.

Необходимые леммы: пояснение



Венгерский алгоритм

Представим граф в виде матрицы весов. Тогда используя описанные выше леммы, можно построить следующий алгоритм:

1. Вычитаем из каждой строки значение ее минимального элемента. Теперь в каждой строке есть хотя бы один нулевой элемент.
2. Вычитаем из каждого столбца значение его минимального элемента. Теперь в каждом столбце есть хотя бы один нулевой элемент.

Венгерский алгоритм

3. Ищем в текущем графе полное паросочетание из ребер нулевого веса:
- Если оно найдено, то желаемый результат достигнут, алгоритм закончен.
 - В противном случае покроем нули матрицы весов минимальным количеством строк и столбцов (это не что иное, как нахождение минимального вершинного покрытия в двудольном графе).

Пусть X_c и Y_c — множества вершин минимального вершинного покрытия из левой и правой долей (то есть строк и столбцов) соответственно, тогда применим преобразование $X_c \uparrow \downarrow Y \setminus Y_c$. Для этого преобразования d будет минимумом по всем ребрам между $X \setminus X_c$ и $Y \setminus Y_c$, то есть ребер нулевого веса здесь нет. Поэтому после его выполнения в матрице весов появится новый нуль. После этого перейдем к шагу 1.

Венгерский алгоритм: пример

2	10	9	7
15	4	14	8
13	14	16	11
4	15	13	19

Венгерский алгоритм: пример

2	10	9	7	-2
15	4	14	8	-4
13	14	16	11	-11
4	15	13	19	-4

Венгерский алгоритм: пример

0	8	7	5	-2
11	0	10	4	-4
2	3	5	0	-11
0	11	9	15	-4

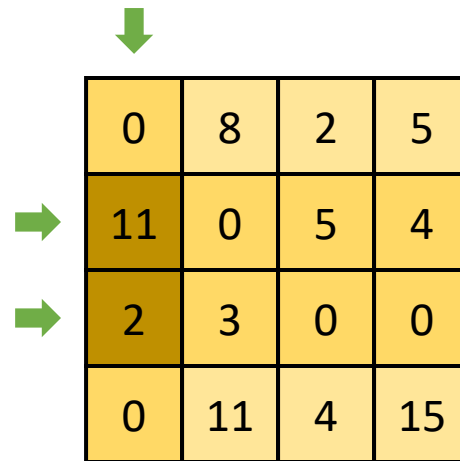
Венгерский алгоритм: пример

0	8	7	5
11	0	10	4
2	3	5	0
0	11	9	15
-0	-0	-5	-0

Венгерский алгоритм: пример

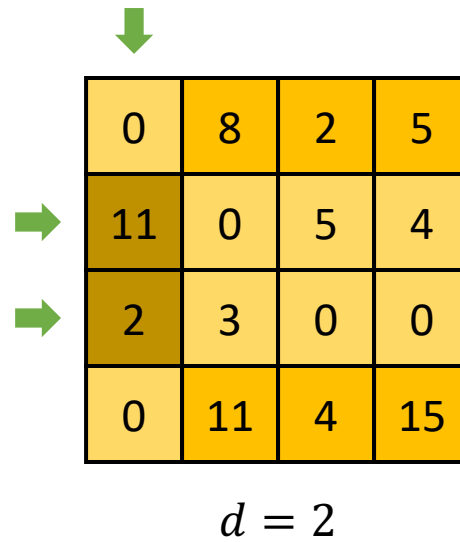
0	8	2	5
11	0	5	4
2	3	0	0
0	11	4	15
-0	-0	-5	-0

Венгерский алгоритм: пример



	0	8	2	5
→	11	0	5	4
→	2	3	0	0
	0	11	4	15

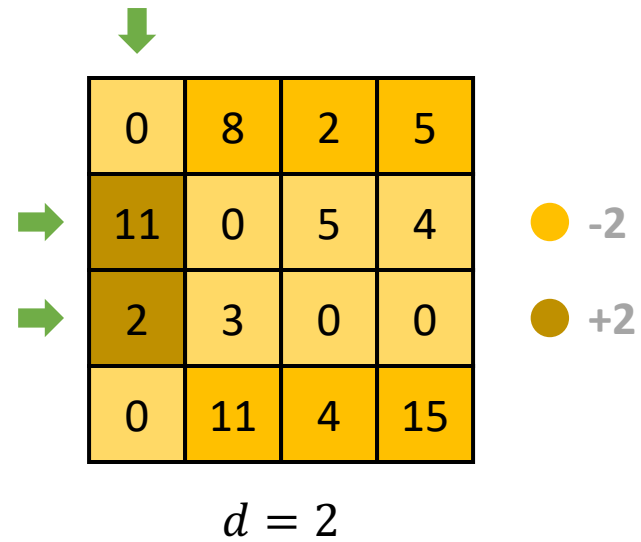
Венгерский алгоритм: пример



	0	8	2	5
→	11	0	5	4
→	2	3	0	0
	0	11	4	15

$d = 2$

Венгерский алгоритм: пример

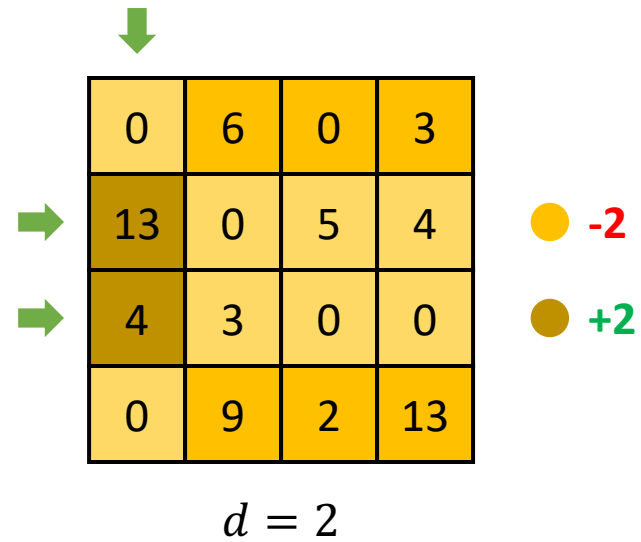


	0	8	2	5
→	11	0	5	4
→	2	3	0	0
	0	11	4	15

$d = 2$

● -2
● +2

Венгерский алгоритм: пример



The diagram illustrates a step in the Hungarian algorithm. A 4x4 cost matrix is shown with yellow cells. A green arrow points down to the first row, and two green arrows point right to the first and third rows. To the right of the matrix, a yellow circle is labeled '-2' and a brown circle is labeled '+2'. Below the matrix, the text $d = 2$ is displayed.

0	6	0	3
13	0	5	4
4	3	0	0
0	9	2	13

$d = 2$

Венгерский алгоритм: пример

0	6	0	3
13	0	5	4
4	3	0	0
0	9	2	13

Венгерский алгоритм: пример

0	6	0	3
13	0	5	4
4	3	0	0
0	9	2	13

Венгерский алгоритм: пример

2	10	9	7
15	4	14	8
13	14	16	11
4	15	13	19

$ans = 28$

Венгерский алгоритм: время работы

Поиск минимального вершинного покрытия работает за $O(n^3)$. На каждой итерации появляется хотя бы один ноль. Так как всего рёбер $O(n^2)$, то и итераций будет $O(n^2)$. Тогда итоговая асимптотика $O(n^5)$.